

FASE 0 · O AGENTE EM CÓDIGO PURO

Aula 0.7

Observabilidade desde o berço

Cada execução vira uma árvore rastreada, com custo em tokens e latência por passo. É o que separa uma demo de uma operação.

Série: Engenharia de IA Aplicada · Agentes e RAG em Produção

Projeto: Núcleo de Agentes Operacionais

Duração de estudo: cerca de 1h30 · 25 min de teoria e 55 de código

Pré-requisito: Aulas 0.1 a 0.6 (tag **aula-0.6**)

1. Objetivo de aprendizagem

Ao final desta aula cada execução do seu agente produz uma árvore rastreada, com duração por passo, tokens por chamada ao modelo e os argumentos de cada ferramenta, permitindo responder depois o que aconteceu e quanto custou.

2. Por que esta aula importa

A pergunta chega numa segunda-feira de manhã e é razoável: por que o sistema abriu três tickets para a mesma pessoa no sábado?

Você abre o terminal e encontra o que há para encontrar: nada. Os `print` rolaram para fora do buffer faz dois dias. Não há como saber o que o agente leu, o que decidiu, quantas vezes chamou o quê. A resposta honesta ao cliente seria "não sei e não tenho como saber", e essa frase, dita sobre um sistema em produção, encerra contratos.

A diferença entre uma demo e uma operação é exatamente a capacidade de responder perguntas sobre execuções que já aconteceram. E são três perguntas, sempre as mesmas: **o que aconteceu, quanto custou e a mudança de ontem melhorou ou piorou.**

2.1 Por que esta aula está aqui e não no fim

Observabilidade quase sempre é ensinada como assunto de produção, junto com deploy. É um erro, e é caro.

Instrumentar um sistema depois de pronto significa revisitar cada arquivo, decidir o que registrar sem lembrar do que importava, e descobrir que a informação mais útil se perdeu porque ninguém a guardou. Instrumentar enquanto se constrói custa dez minutos por aula.

Há também um ganho imediato: a partir daqui você **vê** o laço da aula 0.3 acontecendo. Cada decisão, cada ferramenta, cada token.

3. Teoria

3.1 O vocabulário: trace, span, generation

Três conceitos, e eles são padrão da indústria (vêm do OpenTelemetry), não invenção de nenhuma ferramenta.

Conceito	O que é	No seu código
Trace	Uma execução inteira, de ponta a ponta. A unidade que você abre para investigar um caso.	uma chamada de <code>rodar_agente</code>
Span	Um passo dentro do trace, com começo, fim e duração. Spans aninham, e é esse aninhamento que forma a árvore.	cada execução de ferramenta
Generation	Um span especial: uma chamada ao modelo. Além da duração, carrega o modelo e os tokens, e portanto o custo.	cada volta do laço

A árvore que aparece na tela é literalmente o desenho da aula 0.3 executando.

3.2 Token é a única régua de custo que existe

Na aula 0.3 você aprendeu que o histórico é reenviado inteiro a cada volta. Agora você vai **ver** isso em números: numa execução de quatro voltas, os tokens de entrada crescem a cada chamada, porque cada uma carrega tudo que veio antes.

Por isso o número de iterações é métrica de custo, não de estilo. E por isso os tokens sobem pela árvore no resumo: o custo de uma sessão é a soma de todas as voltas, nunca o da última.

3.3 A exceção consciente à regra do crescimento

Desde a aula 0.1 cada aula só acrescentou arquivos. Esta é a única da fase que faz adições **dentro** de arquivos existentes, e vale entender por quê.

Observabilidade é uma preocupação **transversal**. Ela não é uma etapa do fluxo: ela atravessa todas. Não existe forma de instrumentar o laço sem tocar no laço.

O que se pode fazer, e é o que fazemos, é **confinar a dependência**. Toda a lógica de tracing vive em um arquivo. Os outros ganham chamadas às nossas funções, nunca ao SDK da plataforma. Se um dia você trocar de ferramenta, muda um arquivo. É o mesmo princípio da aula 0.1, aplicado a outra coisa que varia.

3.4 Duas camadas, e a razão de existirem

A instrumentação desta aula tem um rastreador **local**, sempre ligado, que monta a árvore em memória, e uma ponte **opcional** com a plataforma externa.

Isso resolve três problemas de uma vez. Você acompanha a aula sem conta em lugar nenhum. A instrumentação fica testável sem rede, o que importa para a suíte rápida continuar gratuita. E o sistema roda normalmente quando as credenciais não existem, porque **observabilidade nunca pode ser requisito duro para o projeto subir**.

3.5 Uma nota honesta sobre versões

O SDK do Langfuse passou por uma reescrita entre versões maiores: a v3 é construída sobre OpenTelemetry e usa `get_client()` com `start_as_current_observation()`, enquanto a v2 usava `langfuse.trace()`, `langfuse.span()` e `langfuse.generation()`. Há ainda um caminho de atualização para versões posteriores.

Este é o ponto do curso que mais depende de data. Confirme a versão instalada antes de seguir qualquer exemplo, inclusive os deste material. E note que o conceito não depende da ferramenta: Langfuse, LangSmith e qualquer stack baseada em OpenTelemetry usam o mesmo vocabulário. Entendida a árvore, trocar de plataforma é trabalho de tradução.

4. Mão na massa

Passo 1 — O nó da árvore

O que e por quê: uma estrutura de dados que guarda um passo e seus filhos. É ela que torna a árvore inspecionável em teste, e não só visível numa interface web.

```
@dataclass
class Passo:
    """Um nó da árvore de execução."""
    nome: str
```

```

tipo: str # "trace", "span" ou "generation"
entrada: object = None
saida: object = None
inicio: float = 0.0
fim: float = 0.0
modelo: str = ""
tokens_entrada: int = 0
tokens_saida: int = 0
filhos: list = field(default_factory=list)

@property
def duracao_ms(self):
    return int((self.fim - self.inicio) * 1000)

def totais(self):
    """Soma tokens da subarvore inteira, recursivamente."""
    entrada = self.tokens_entrada
    saida = self.tokens_saida
    for f in self.filhos:
        e, s = f.totais()
        entrada += e
        saida += s
    return entrada, saida

```

O que faz: `totais()` soma os tokens da subárvore recursivamente. É a implementação literal da regra da seção 3.2.

Passo 2 — Abrir e fechar passos

O que e por quê: um gerenciador de contexto que empilha o passo ao entrar e desempilha ao sair. O aninhamento sai de graça: cada passo aberto vira filho do que estiver no topo da pilha.

```

_pilha: ContextVar = ContextVar("pilha_de_passos", default=None)

@contextmanager
def observar(nome, tipo="span", entrada=None, modelo=""):
    """Abre um passo e o fecha ao sair do bloco.

    O aninhamento sai de graça: cada passo aberto vira filho do que
    estiver no topo da pilha. E isso que forma a arvore.
    """
    pilha = _pilha.get()
    if pilha is None:
        # Ninguem abriu um trace: nao rastreamos, mas o codigo roda.
        yield None
        return

    passo = Passo(nome=nome, tipo=tipo, entrada=entrada,
                  modelo=modelo, inicio=time.time())
    if pilha:
        pilha[-1].filhos.append(passo)
    pilha.append(passo)

    # A ponte com a plataforma externa, quando existir, abre o mesmo
    # passo la. Nada do resto do Nucleo precisa saber disso.
    externo = _abrir_externo(nome, tipo, entrada)
    try:
        yield passo
    finally:
        passo.fim = time.time()
        pilha.pop()

```

```

        _fechar_externo(externo, passo)

@contextmanager
def iniciar_trace(nome, entrada=None):
    """Abre a execucao inteira. Tudo que acontecer dentro vira filho."""
    token = _pilha.set([])
    try:
        with observar(nome, tipo="trace", entrada=entrada) as raiz:
            yield raiz
    finally:
        _pilha.reset(token)
        _enviar_pendentes()

```

O que faz: note o `ContextVar` em vez de variável global. Quando o Núcleo estiver atrás de uma API na Fase 4, atendendo várias requisições ao mesmo tempo, cada contexto vai enxergar o próprio trace. Decisão barata agora, cara depois.

E note o `yield None` quando não há trace aberto: código instrumentado precisa rodar em qualquer contexto, inclusive num teste que não abriu trace nenhum.

Passo 3 — Instrumentar o laço

O que e por quê: a parte transversal. Cada volta vira uma generation, cada ferramenta vira um span.

```

# DECISAO: o modelo le o contexto e escolhe o proximo passo.
# Chamada ao modelo e GENERATION: e nela que mora o custo.
with observar(f"decisao {iteracao}", tipo="generation") as gen:
    resposta = chat_ferramentas(
        provider, historico, ferramentas, system=system)
    registrar_uso(gen, resposta.tokens_entrada,
                  resposta.tokens_saida, resposta.modelo)

```

```

# Execucao de ferramenta e SPAN: tem duracao, nao tem
# token. E aqui que se descobre o agente chamando a
# mesma coisa tres vezes.
with observar(pedido.nome, entrada=pedido.argumentos) as sp:
    funcao = executores.get(pedido.nome)
    if funcao is None:
        # Ferramenta desconhecida NAO derruba o agente:
        # vira observacao que ele le e contorna.
        saida = {"erro": f"ferramenta desconhecida: "
                    f"{pedido.nome}"}
    else:
        saida = funcao(**pedido.argumentos)
    if sp is not None:
        sp.saida = saida

```

O que faz: a chamada ao modelo registra os tokens, que a camada de provider agora normaliza (`input_tokens` na Anthropic, `prompt_tokens` na OpenAI, mesma diferença morrendo no mesmo lugar de sempre). A ferramenta registra argumentos e resultado, e é assim que você descobre, na segunda-feira, com que argumentos ela foi chamada no sábado.

Passo 4 — A ponte com a plataforma

O que e por quê: o único trecho acoplado a um fornecedor específico, isolado num canto.

```
def _cliente():
    if not TRACING_ATIVO:
        return None
    try:
        from langfuse import get_client
        return get_client()
    except Exception:
        # Pacote ausente ou credencial invalida: seguimos sem tracing
        # externo. O rastreador local continua funcionando.
        return None
```

O que faz: import tardio e falha silenciosa. Pacote ausente ou credencial inválida não derrubam nada; o rastreador local segue funcionando.

Passo 5 — O flush, que é a causa número um de "não funcionou"

```
def _enviar_pendentes():
    """Força o envio do lote antes do processo morrer.

    CAUSA NUMERO UM de "o trace nao apareceu": as plataformas enviam
    em lote, de forma assincrona, e script curto termina antes do
    envio. Em servidor de longa duracao isso nao aparece, o que torna
    o problema ainda mais confuso quando volta.
    """
    cliente = _cliente()
    if cliente is not None:
        try:
            cliente.flush()
        except Exception:
            pass
```

GUARDE ISTO

As plataformas de tracing enviam eventos **em lote, de forma assíncrona**, para não adicionar latência. Em script curto, o processo termina antes do envio e o trace nunca aparece. Em servidor de longa duração o problema não aparece, o que o torna ainda mais confuso quando volta.

Passo 6 — Ver funcionando

```
# scratch/rascunho_0_7.py (descartavel)
from nucleo import banco, ferramentas
from nucleo.registro import schemas
from nucleo.resiliencia import executores_seguros
from nucleo.observabilidade import arvore, resumo
from nucleo.agente import rodar_agente

banco.criar_schema()

SISTEMA = """Voce e o assistente comercial do FlowDesk, um SaaS B2B.
Consulte o CRM antes de agir. Problema tecnico vira ticket."""

exe = rodar_agente("A Ana do TechFlow relatou erro 500 no login.",
                  schemas(), executores_seguros(), system=SISTEMA,
                  verbose=False)

print(exe.resposta)
print()
print(arvore(exe.trace))      # a execucao inteira, em arvore
```

```
print()
print(resumo(exe.trace))      # duracao, chamadas e tokens
```

Resultado esperado, numa execução em que o modelo erra a prioridade e se recupera (a resiliência da aula 0.6, agora visível):

```
[TRACE] rodar_agente 8431ms
[gen] decisao 1 1902ms (1180 in / 62 out)
[span] buscar_atualizar_crm 3ms
[gen] decisao 2 2140ms (1420 in / 88 out)
[span] abrir_ticket 1ms
[gen] decisao 3 2311ms (1690 in / 85 out)
[span] abrir_ticket 4ms
[gen] decisao 4 2075ms (1810 in / 31 out)

resumo: {'duracao_ms': 8431, 'chamadas_ao_modelo': 4,
        'ferramentas_executadas': 3, 'tokens_entrada': 6100,
        'tokens_saida': 266}
```

Leia os números. O `abrir_ticket` aparece **duas vezes**: é o agente corrigindo o argumento inválido. E os tokens de entrada crescem a cada volta, de 1180 para 1810, porque cada chamada carrega o histórico inteiro. Antes desta aula, nada disso era visível.

Passo 7 — Os testes

O que e por quê: nove testes determinísticos. Dois merecem destaque. O que prova a regra do custo:

```
# ----- 3. tokens sobem pela arvore -----

def test_tokens_somam_na_subarvore():
    with iniciar_trace("execucao") as raiz:
        with observar("c1", tipo="generation") as g1:
            registrar_uso(g1, 1000, 50)
        with observar("c2", tipo="generation") as g2:
            registrar_uso(g2, 1500, 80)

    entrada, saida = raiz.totais()
    assert entrada == 2500
    assert saida == 130
    # 0 custo de uma sessao e a soma de TODAS as voltas, nao da ultima.
```

E o que garante a informação de depuração que você vai querer na segunda-feira:

```
# ----- 8. a ferramenta registra argumentos e resultado -----

def test_span_de_ferramenta_guarda_entrada_e_saida(monkeypatch):
    fila = [
        RespostaLLM("", [PedidoFerramenta("1", "buscar_info_lead",
                                           {"email": "c@d.io"})]),
        {"role": "assistant", "content": []}],
        RespostaLLM("ok", []),
    ]
    monkeypatch.setattr(agente, "chat_ferramentas",
                        lambda *a, **k: fila.pop(0))
    monkeypatch.setattr(agente, "mensagens_de_resultado",
                        lambda p, r, res: [{"role": "user", "content": "x"}])

    exe = agente.rodar_agente("?", FER, _executores([]), verbose=False)
    span = [f for f in exe.trace.filhos if f.nome == "buscar_info_lead"][0]
```

```
assert span.entrada == {"email": "c@d.io"}
assert span.saida == {"plano": "pro"}
# E assim que se descobre, na segunda-feira, com que argumentos o
# agente chamou a ferramenta no sabado.
```

Resultado esperado: `pytest` mostra `52 passed, 2 deselected`.

Passo 8 — Commit e tag

```
pytest -v          # esperado: 52 passed, 2 deselected
git status         # .env, scratch/ e *.db NAO podem aparecer
git add .
git commit -m "Aula 0.7: observabilidade desde o berco"
git tag aula-0.7
git push && git push --tags
```

5. Portabilidade

O vocabulário de trace, span e atributos é do OpenTelemetry, o padrão aberto de observabilidade. Langfuse, LangSmith e as stacks corporativas falam a mesma língua.

Na Fase 1, o LangGraph emite esses eventos sozinho: cada node vira um span automaticamente. Você não vai escrever mais a instrumentação à mão, mas vai saber exatamente o que está vendo, e vai saber o que falta quando faltar.

E o padrão de confinar a dependência num arquivo é o mesmo da aula 0.1: isole o que varia atrás de uma interface que não varia. Vale para provider, vale para banco, vale para plataforma de tracing.

6. Terminei quando

O critério da aula: **eu vejo a qualificação inteira rastreada, com o custo daquela sessão.**

- A árvore mostra cada decisão e cada ferramenta, aninhadas e com duração.
- Cada chamada ao modelo mostra tokens de entrada e saída, e você consegue explicar por que a entrada cresce a cada volta.
- Você olha a árvore de uma execução que falhou e aponta em qual passo ela começou a dar errado.
- O sistema roda normalmente sem credencial de tracing configurada.
- `pytest` mostra `52 passed, 2 deselected`.

7. Erros comuns e troubleshooting

- **O trace não aparece na plataforma.** Em 90% dos casos, falta o flush: o processo terminou antes do envio em lote.
- **A árvore sai achatada, sem aninhamento.** Alguém abriu um passo fora do bloco do trace, ou fechou o trace antes da hora. O aninhamento depende da pilha.
- **Tokens zerados em todas as chamadas.** O dublê de teste não tem `usage`, e isso é esperado. Em chamada real, confira se o campo existe na resposta do provider que você está

usando.

- **ImportError do pacote de tracing.** O import é tardio e protegido justamente para isso. Se estiver estourando, alguém importou no topo do arquivo.
- **A suíte ficou lenta.** Alguma credencial de tracing está configurada no ambiente de teste, e os testes passaram a enviar eventos. Teste não deve falar com plataforma externa.
- **Dados sensíveis no trace.** Você está registrando a entrada e a saída de cada ferramenta, e isso inclui o que o cliente digitou. Em produção, isso é decisão de privacidade, e volta a aparecer na Fase 5.

8. Ponte para a próxima aula

A Fase 0 está completa em capacidade: o agente age, resiste a falha, devolve contrato e é observável. Falta arrumar a casa. Na **Aula 0.8, Consolidação**, o repositório vira portfólio, e você vai aprender a apontar a fronteira entre o que é conhecimento portátil e o que vira framework na Fase 1. Saber apontar essa fronteira é o que separa quem usa framework de quem escolhe framework.